RUTGERS UNIVERSITY · FALL 2026

CS 110

Sequential Programming

Recitation 6



Antara Thapliyal · Recitation Instructor

Sections 1 & 2 · ARC 116 (Busch)

First: Exam Questions

Five minutes for anything you want to ask about the exam.

Ask me now

Format, topics, what to review, how the second chance policy works — anything.

Exam dates

Hourly Exam 1 is Thursday, October 8.
Exam 2 is Thursday, November 12.

Second chance

A stronger score on the matching part of the final replaces an hourly exam grade.

Exam grade questions

Those go to your lecture instructor, not to me, within two weeks.

Today’s Plan

Writing and running programs together. Open your IDE now.

Exam questions	Anything you want to ask before we start.	**5 min**
The building blocks	Input, casting, division, and tidy output.	**15 min**
Seven programs	We write each one together, then run it.	**30 min**
Wrap up	What to practise before the assignment.	**5 min**

Type along with me. You learn this by making the mistakes yourself, not by watching me avoid them.

PART ONE

The Building Blocks

Four things every program today uses.

About 15 minutes



Two Rules for Today

Top to bottom

Python runs your lines in order, first to last, once each. Nothing is skipped and nothing repeats. That is what “sequential” means.

No if statements

Not even for the True/False problems. If you find yourself wanting one, you are looking for a Boolean expression instead — we will see exactly how.

Conditionals come later in the course. Today is about getting comfortable writing straight-line code.

input() Always Gives You Text

Even when the user types a number, Python hands you a string.

```
agesStr = input("How old are you? ")    # text
ages = int(agesStr)                     # a number
```

The naming convention we use all semester

1. Read the input into a name ending in Str.
2. Cast it to int or float.
3. Store the result under the same name, without the Str.

Why it matters

"5" + "3" gives 53, not 8. Text joins; numbers add. Forget to cast and your arithmetic silently goes wrong.

int or float?

- int — whole things you cannot have half of: people, days, guests.
- float — anything that can have a decimal: miles, gallons, degrees.

Three Kinds of Division

Slash, double slash, and percent do three different jobs.

/

True division

`7 / 2  →  3.5`

Always gives a decimal, even when it divides evenly.

//

Floor division

`7 // 2  →  3`

Throws away the remainder. How many whole times it fits.

%

Modulo

`7 % 2  →  1`

Gives only the remainder. The part left over.

Together they split a quantity up: 3725 seconds

```
3725 // 3600  =  1 hour

3725  % 3600  =  125 seconds left over

 125 // 60    =  2 minutes,   125 % 60  =  5 seconds
```

Know these: 60 seconds in a minute, 3600 seconds in an hour.

Making Output Readable

`print("")`	Prints a blank line — space out your output.
`print("\n Title")`	The \n skips a line before the text.
`print("\t Indented")`	The \t inserts a tab, so the line is indented.
`round(value, 2)`	Rounds to two decimal places.
`round(value)`	Rounds to the nearest whole number.

math.ceil and math.floor

You need these for the assignment.

```
import math
print(math.ceil(4.2))
print(math.floor(4.9))
```

ceil always rounds UP: 4.2 becomes 5.
floor always rounds DOWN: 4.9 becomes 4.
Neither cares which is nearer.

PART TWO

Seven Programs

We write each one together. Type along.

About 30 minutes



Problem 1: Fahrenheit to Celsius

Convert a temperature the user types in.

```
# degreesC = (degreesF - 32) * 5/9
degreesFStr = input("Enter degrees Fahrenheit: ")
degreesF = float(degreesFStr)
degreesC = (degreesF - 32) * 5/9
print("Celsius degrees =", round(degreesC, 2))
```

Sample run

```
Enter degrees
Fahrenheit: 98.6

Celsius degrees = 37.0
```

Problem 2: Miles Per Gallon

Ask for miles driven and gallons used, then print a message with the answer.

```
milesStr = input("How many miles did you drive? ")
miles = float(milesStr)

galsUsedStr = input("How many gallons did you use? ")
galsUsed = float(galsUsedStr)

mpg = miles / galsUsed
print("You got", round(mpg, 2), "miles to the gallon")
```

Sample run

```
...miles did you drive? 300
...gallons did you use? 12

You got 25.0 miles
to the gallon
```

Problem 3: Is the Venue Within Capacity?

Print True or False — without an if statement.

```
capacityStr = input("Enter the venue capacity: ")
capacity = int(capacityStr)    # people, so int

guestsStr = input("How many guests have arrived? ")
guests = int(guestsStr)

withinLimits = guests <= capacity
print("Within limits?", withinLimits)
```

Sample run

```
...venue capacity: 150
...guests arrived? 162

Within limits? False
```

The key line

guests <= capacity is a question. Python answers True or False, and you can store that answer in a variable like any other value.

Problem 4: What Day Will You Return?

Days are numbered 0 to 6, where 0 is Sunday. Leave on day 3, stay 10 nights — you return on day 6.

```
startDayStr = input("What day number do you leave? ")
startDay = int(startDayStr)

numDaysStr = input("How many days will you stay? ")
numDays = int(numDaysStr)

returnDay = (startDay + numDays) % 7
print("You will return on day", returnDay)
```

Sample run

```
...day do you leave? 3
...will you stay? 10

You will return on day 6
```

Why % 7 works

3 + 10 = 13. The week only has 7 days, so 13 % 7 = 6. Modulo wraps you back around.

Problem 5: Area of a Rectangle

Same program, written more compactly — casting inside input, and the calculation inside print.

```
# rectangle
length = float(input("Enter rectangle length: "))
width = float(input("Enter rectangle width: "))

print("The area is:", length * width)
```

Sample run

```
...rectangle length: 8
...rectangle width: 4.5

The area is: 36.0
```

Two styles, same result

Problems 1–4 used a separate line for each step. This one folds them together. Both are correct — the step-by-step version is easier to debug when something goes wrong.

Problem 6: Even or Odd?

Print True if the integer is even, False if it is odd. Still no if statement.

```
# another Boolean expression, using %
num = int(input("Enter an integer: "))

even = num % 2 == 0
print("Is it even?", even)
```

Sample run

```
Enter an integer: 7

Is it even? False
```

Problem 7: Accumulating a Total

Start a total at 0, add three numbers to it one at a time, then print the total and the average.

```
total = 0

num1 = int(input("Enter num1: "))
total = total + num1

num2 = int(input("Enter num2: "))
total = total + num2

num3 = int(input("Enter num3: "))
total = total + num3

print("The total is:", total)
print("The average is:", round(total / 3, 2))
```

Sample run

```
Enter num1: 10
Enter num2: 15
Enter num3: 20

The total is: 45
The average is: 15.0
```

total = total + num1 is not algebra. It means: work out the right side, then store that answer back in total.

Before the Assignment

The Str naming convention	Read into nameStr, cast, store as name. Use it every time.
Cast before you calculate	input() gives text. Text does not do arithmetic.
Booleans instead of if statements	A comparison is a value. Store it, print it.
/ versus // versus %	Decimal, whole part, remainder. Know which one you need.
math.ceil and math.floor	Always up, always down. You will need both.

Retype these programs yourself this week. Reading code is not the same as writing it.

Questions?

Bring them to recitation, office hours, lab support, or Piazza.



at1679@scarletmail.rutgers.edu

Office hours: Thursdays 11:00 AM – 12:00 PM (Zoom link on Canvas)

Lab support: Wednesdays 12:00 – 2:00 PM, ARC (Busch)